Guía del notebook `1_09_red_multicapa_ejercicio`: una red con capa oculta

Puntos: 3 · **Tiempo:** 20–25 min · **Módulo:** 4, redes multicapa (clase 5) · **Archivo:** `notebooks/1_09_red_multicapa_ejercicio.ipynb`
Lectura: libro, Redes neuronales multicapa.

¿Qué vas a aprender?

- Qué es una **capa oculta** y por qué hace falta.
- A representar los pesos de cada capa como una **matriz**.
- A calcular la salida de una red de dos capas (**pase hacia adelante**) con productos de matrices.

Antes de empezar

1. `conda activate nni`.
2. VS Code → **Open Folder** → `curso-redes-neuronales` → `notebooks/1_09_red_multicapa_ejercicio.ipynb` → kernel **nni**.
   El notebook usa el paquete `nni` y la imagen `files/test1.png` de la carpeta `notebooks`.
3. Ejecuta las celdas en orden con **Shift + Enter**.

La idea en 2 minutos

¿Por qué capas ocultas?

Una sola neurona solo traza **una recta**, y por eso no puede con XOR (Clases 2 y 4). Si ponemos una **capa de neuronas en medio** (la capa oculta), cada una traza su recta y la neurona de salida las combina. Así la red puede separar regiones con formas más complejas. Esta red también se llama **perceptrón multicapa** (MLP).

La red de este notebook

4 entradas → 3 neuronas ocultas → 2 salidas, como en la figura del notebook.

Pesos como matrices

- Entre la entrada (4) y la oculta (3) hay 4 × 3 = 12 pesos. Van en una matriz **W_1 de 4 filas y 3 columnas**: las filas son las entradas y las columnas, las neuronas de la capa siguiente.
- Entre la oculta (3) y la salida (2) hay una matriz **W_2 de 3 × 2**.

Pase hacia adelante (dos veces lo mismo de la Clase 4)

Paso	Fórmula	Tamaño del resultado
Capa oculta	$H_1 = X \cdot W_1 \ (+ B_1)$ y $A_1 = \sigma(H_1)$	1 × 3
Salida	$H_2 = A_1 \cdot W_2 \ (+ B_2)$ y $\hat{Y} = \sigma(H_2)$	1 × 2

La salida de la capa oculta, $\mathbf{A_1}$, es la entrada de la siguiente.

Regla de los tamaños: para multiplicar (a × b)·(b × c), el número del medio debe coincidir, y el resultado es (a × c). Por ejemplo, (1 × 4)·(4 × 3) = (1 × 3).

Paso a paso

Celda 1: paquetes

Importa NumPy y `nni`, y fija la semilla 42.

Celda: tamaños de las capas

Completa `N_input`, `N_hidden` y `N_output` según la figura: **4, 3 y 2**.

Celda `TODO (1 punto)`: pesos y entrada

1. **Media y desviación:** media 0 y una desviación pequeña. Una buena elección es **1/√(número de entradas)**, la inicialización normal escalada que vimos con Xavier.
2. **Matrices:** `np.random.normal(mean, stdev, size=(N_input, N_hidden))` para W_1, y de tamaño `(N_hidden, N_output)` para W_2. Usa las variables de tamaño, no números fijos.
3. **Entrada:** un vector aleatorio de tamaño `(1, N_input)`, por ejemplo con `np.random.normal(size=(1, N_input))`.

✅ **Comprobación:** imprime las formas. Deben ser W_1 **(4, 3)**, W_2 **(3, 2)** y X **(1, 4)**.

Celda `TODO (1 punto)`: calcular la salida

- `H_1` = producto de X por W_1 (`np.dot`).
- `H_2` = producto de A_1 por W_2. **Ojo:** se usa A_1 (después de la sigmoide), no H_1.

✅ **Comprobación:**

- la salida de la capa oculta tiene **3 números** entre 0 y 1;
- la salida de la red tiene **2 números** entre 0 y 1.

Con la semilla 42, y generando W_1, W_2 y X en ese orden con desviación 1/√4, sale:

- capa oculta ≈ [0.4489, 0.6991, 0.3965];
- red ≈ [0.3210, 0.3627].

Si generas en otro orden, los números cambian, pero los tamaños no.

Último punto (1 punto): sesgos

Agrega una celda nueva con un vector de sesgos para cada capa: **B_1 de tamaño (1, 3)** y **B_2 de tamaño (1, 2)**. Súmalos antes de cada sigmoide.

✅ **Comprobación:**

- con sesgos en cero, la salida es igual a la de antes;
- con otros valores, cambia;
- en total la red tiene 4·3 + 3 + 3·2 + 2 = **23 parámetros**.

Errores frecuentes

Síntoma	Causa	Solución
`ValueError: shapes (1,4) and (3,2) not aligned`	Multiplicaste X por W_2, o H_1 en lugar de A_1	Sigue el orden: X·W_1 → σ → A_1·W_2 → σ
`TypeError: ... NoneType`	Algún tamaño o matriz sigue en `None`	Completa todas las variables
`AttributeError: module 'nni' has no attribute 'sigmoid'`	Notebook abierto fuera del repo, o `nni` viejo	Abre la carpeta `curso-redes-neuronales` y haz `git pull`
Salidas iguales a 0.5	Pesos o entradas todos en cero	Usa `np.random.normal`